

# Parameter Table Forwarder

## *Component Design Document*

### 1 Description

Sits between an upstream parameter table source (Ccsds\_Parameter\_Table\_Router or an upstream Parameter\_Store) and a single downstream component that owns its own parameter table layout. Validates incoming memory regions (length and CRC), forwards all parameter table operations (Set, Validate, Get) to the downstream component, and provides standard boilerplate (dump command, active parameters packet, table status data product, common events) so the downstream component only has to deserialize and apply its own packed record. The forwarder owns no parameter table bytes – it is pure middleware. The downstream component is the authoritative source of the table contents and is queried via the Get operation when a dump is requested.

### 2 Requirements

The requirements for the Parameter Table Forwarder component are specified below.

1. The component shall forward Set parameter table operations from an upstream source to a downstream component after validating length and CRC.
2. The component shall forward Validate parameter table operations from an upstream source to a downstream component after validating length and CRC.
3. The component shall forward Get parameter table operations from an upstream source to a downstream component after validating length.
4. The component shall release the incoming memory region back to the upstream source with a status reflecting the downstream component's response or the forwarder's own validation failure.
5. The component shall produce an active parameters packet on command, sourced from the downstream component via an internal Get operation.
6. The component shall produce an active parameters packet automatically after a successful Set when Dump\_Parameters\_On\_Change is True.
7. The component shall reject Set operations whose memory region length does not equal the configured table size.
8. The component shall reject Set operations whose stored CRC does not match the computed CRC.
9. The component shall reject Get operations whose memory region length is less than the configured table size.
10. The component shall produce a Table\_Status data product reflecting the latest table operation, including version, CRC, timestamp, and operation status.

## 3 Design

### 3.1 At a Glance

Below is a list of useful parameters and statistics that give a quick look into the makeup of the component.

- **Execution** - *active*
- **Number of Connectors** - 9
- **Number of Invokee Connectors** - 2
- **Number of Invoker Connectors** - 7
- **Number of Generic Connectors** - *None*
- **Number of Generic Types** - *None*
- **Number of Unconstrained Arrayed Connectors** - *None*
- **Number of Commands** - 1
- **Number of Parameters** - *None*
- **Number of Events** - 14
- **Number of Faults** - *None*
- **Number of Data Products** - 1
- **Number of Data Dependencies** - *None*
- **Number of Packets** - 1

### 3.2 Diagram

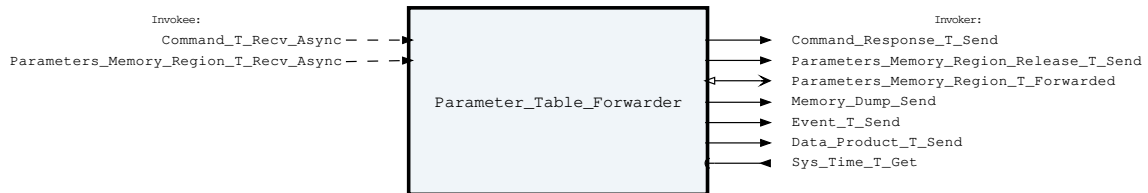


Figure 1: Parameter Table Forwarder component diagram.

### 3.3 Connectors

Below are tables listing the component's connectors.

#### 3.3.1 Invokee Connectors

The following is a list of the component's *invokee* connectors:

Table 1: Parameter Table Forwarder Invokee Connectors

| Name                                  | Kind       | Type                       | Return_Type | Count |
|---------------------------------------|------------|----------------------------|-------------|-------|
| Command_T_Recv_Async                  | recv_async | Command.T                  | -           | 1     |
| Parameters_Memory_Region_T_Recv_Async | recv_async | Parameters_Memory_Region.T | -           | 1     |

---

Connector Descriptions:

- **Command\_T\_Recv\_Async** - Command receive.
- **Parameters\_Memory\_Region\_T\_Recv\_Async** - Inbound parameter table memory region from the router (or an upstream Parameter\_Store). The operation (Set, Validate, Get) is in the region.

### 3.3.2 Internal Queue

This component contains an internal first-in-first-out (FIFO) queue to handle asynchronous messages. This queue is sized at initialization as a configurable number of bytes. Determining the size of the component queue can be difficult. The following table lists the connectors that will put asynchronous messages onto the queue, and the maximum sizes of each of those messages on the queue. Note that each message put onto the queue also incurs an overhead on the queue of 5 additional bytes, which is included in the max message size below:

Table 2: Parameter Table Forwarder Asynchronous Connectors

| Name                                  | Type                       | Max Size (bytes) |
|---------------------------------------|----------------------------|------------------|
| Command_T_Recv_Async                  | Command.T                  | 265              |
| Parameters_Memory_Region_T_Recv_Async | Parameters_Memory_Region.T | 18               |

If you are unsure how to size the queue of this component, it is recommended that you make the queue size a multiple of the largest size found above.

### 3.3.3 Invoker Connectors

The following is a list of the component's *invoker* connectors:

Table 3: Parameter Table Forwarder Invoker Connectors

| Name                                    | Kind    | Type                                | Return_Type                        | Count |
|-----------------------------------------|---------|-------------------------------------|------------------------------------|-------|
| Command_Response_T_Send                 | send    | Command_Response.T                  | -                                  | 1     |
| Parameters_Memory_Region_Release_T_Send | send    | Parameters_Memory_Region_Release.T  | -                                  | 1     |
| Parameters_Memory_Region_T_Forwarded    | request | Parameters_Memory_Region.T          | Parameters_Memory_Region_Release.T | 1     |
| Memory_Dump_Send                        | send    | Memory_Packetizer_Types.Memory_Dump | -                                  | 1     |
| Event_T_Send                            | send    | Event.T                             | -                                  | 1     |
| Data_Product_T_Send                     | send    | Data_Product.T                      | -                                  | 1     |
| Sys_Time_T_Get                          | get     | -                                   | Sys_Time.T                         | 1     |

Connector Descriptions:

- **Command\_Response\_T\_Send** - Command response.

- **Parameters\_Memory\_Region\_Release\_T\_Send** - Release back to the upstream sender with the operation status.
- **Parameters\_Memory\_Region\_T\_Forwarded** - Outbound validated memory region forwarded to the downstream component;  
the downstream returns its operation release status synchronously.  
For Set/Validate, the region points at the upstream caller's buffer with the validated header and data. For Get, the region points at the upstream caller's buffer to be filled by the downstream component. For internal Dump operations, the region points at the outgoing packet's buffer.
- **Memory\_Dump\_Send** - Memory dump send connector. The forwarder emits the active parameter table as two Memory\_Dump records (header + payload) under the same Active\_Parameters packet ID. A downstream Memory\_Packetizer chunks each region into Packet.T's that fit Configuration.Packet\_Buffer\_Size, so the table is not bounded by a single Packet.T's capacity.
- **Event\_T\_Send** - Events out.
- **Data\_Product\_T\_Send** - Data products out.
- **Sys\_Time\_T\_Get** - System time.

### 3.4 Interrupts

This component contains no interrupts.

### 3.5 Initialization

Below are details on how the component should be initialized in an assembly.

#### 3.5.1 Component Instantiation

This component contains no instantiation parameters in its discriminant.

#### 3.5.2 Component Base Initialization

This component achieves base class initialization using the `init_Base` subprogram. This subprogram requires the following parameters:

Table 4: Parameter Table Forwarder Base Initialization Parameters

| Name       | Type    |
|------------|---------|
| Queue_Size | Natural |

Parameter Descriptions:

- **Queue\_Size** - The number of bytes that can be stored in the component's internal queue.

#### 3.5.3 Component Set ID Bases

This component contains commands, events, packets, faults, or data products that require a base identifier to be set at initialization. The `set_Id_Bases` procedure must be called with the following parameters:

Table 5: Parameter Table Forwarder Set Id Bases Parameters

| Name                 | Type                                    |
|----------------------|-----------------------------------------|
| Command_Id_Base      | Command_Types.Command_Id_Base           |
| Data_Product_Id_Base | Data_Product_Types.Data_Product_Id_Base |
| Event_Id_Base        | Event_Types.Event_Id_Base               |
| Packet_Id_Base       | Packet_Types.Packet_Id_Base             |

Parameter Descriptions:

- **Command\_Id\_Base** - The value at which the component's command identifiers begin.
- **Data\_Product\_Id\_Base** - The value at which the component's data product identifiers begin.
- **Event\_Id\_Base** - The value at which the component's event identifiers begin.
- **Packet\_Id\_Base** - The value at which the component's unresolved packet identifiers begin.

### 3.5.4 Component Map Data Dependencies

This component contains no data dependencies.

### 3.5.5 Component Implementation Initialization

The calling of this implementation class initialization procedure is mandatory. The forwarder is initialized with the expected size of the downstream component's parameter table and a dump-on-change flag. The `init` subprogram requires the following parameters:

Table 6: Parameter Table Forwarder Implementation Initialization Parameters

| Name                      | Type    | Default Value        |
|---------------------------|---------|----------------------|
| Table_Size                | Natural | <i>None provided</i> |
| Dump_Parameters_On_Change | Boolean | False                |

Parameter Descriptions:

- **Table\_Size** - The expected size in bytes of the parameter table managed by the downstream component. Set/Validate operations require the incoming memory region length to equal `Table_Size` exactly. Get operations require the incoming memory region length to be at least `Table_Size`.
- **Dump\_Parameters\_On\_Change** - If True, automatically dump the current table (by forwarding a Get to the downstream component) after every Set that returns Success.

## 3.6 Commands

These are the commands for the Parameter Table Forwarder component.

Table 7: Parameter Table Forwarder Commands

| Local ID | Command Name         | Argument Type |
|----------|----------------------|---------------|
| 0        | Dump_Parameter_Table | -             |

Command Descriptions:

- **Dump\_Parameter\_Table** - Fetch the current parameter table from the downstream component by issuing an internal Get and emit it as an Active\_Parameters packet.

### 3.7 Parameters

The Parameter Table Forwarder component has no parameters.

### 3.8 Events

Events for the Parameter Table Forwarder component.

Table 8: Parameter Table Forwarder Events

| Local ID | Event Name                               | Parameter Type                            |
|----------|------------------------------------------|-------------------------------------------|
| 0        | Memory_Region_Length_Mismatch            | Invalid_Parameters_Memory_Region_Length.T |
| 1        | Memory_Region_Crc_Invalid                | Invalid_Parameters_Memory_Region_Crc.T    |
| 2        | Parameter_Table_Updated                  | Memory_Region.T                           |
| 3        | Parameter_Table_Validated                | Memory_Region.T                           |
| 4        | Parameter_Table_Fetched                  | Memory_Region.T                           |
| 5        | Downstream_Component_Rejected_Update     | Parameters_Memory_Region_Release.T        |
| 6        | Downstream_Component_Rejected_Validation | Parameters_Memory_Region_Release.T        |
| 7        | Downstream_Component_Rejected_Fetch      | Parameters_Memory_Region_Release.T        |
| 8        | Get_Pointer_Not_Supported                | Memory_Region.T                           |
| 9        | Dumped_Parameters                        | -                                         |
| 10       | Dump_Failed                              | Parameters_Memory_Region_Release.T        |
| 11       | Invalid_Command_Received                 | Invalid_Command_Info.T                    |
| 12       | Command_Dropped                          | Command_Header.T                          |
| 13       | Memory_Region_Dropped                    | Parameters_Memory_Region.T                |

Event Descriptions:

- **Memory\_Region\_Length\_Mismatch** - An incoming memory region length did not match the configured Table\_Size.
- **Memory\_Region\_Crc\_Invalid** - An incoming memory region's stored CRC did not match the computed CRC over its contents.
- **Parameter\_Table\_Updated** - A new parameter table was successfully applied by the downstream component.
- **Parameter\_Table\_Validated** - A parameter table was successfully validated by the downstream component.
- **Parameter\_Table\_Fetched** - The current parameter table was written into the requester's memory region by the downstream component.
- **Downstream\_Component\_Rejected\_Update** - The downstream component rejected a Set operation.

- **Downstream\_Component\_Rejected\_Validation** - The downstream component rejected a Validate operation.
- **Downstream\_Component\_Rejected\_Fetch** - The downstream component rejected a Get operation.
- **Get\_Pointer\_Not\_Supported** - A Get\_Pointer request was received from an external caller. The forwarder owns the table header and strips it on every inbound Set/Validate; it could not insert the header into a caller-visible region returned by the downstream. Callers wanting a full table must use Get\_Copy.
- **Dumped\_Parameters** - An Active\_Parameters packet was emitted.
- **Dump\_Failed** - A dump pathway operation failed. Emitted in three cases: (1) the Dump\_Parameter\_Table command was issued while the Memory\_Dump\_Send connector was unwired (release region uses a 0xDEAD\_BEEF sentinel address with Length 0); (2) the downstream component rejected the internal Get\_Pointer during the Dump\_Parameter\_Table command; (3) the auto-dump triggered after a successful Set when Dump\_Parameters\_On\_Change is True could not complete.
- **Invalid\_Command\_Received** - A command was received with invalid parameters.
- **Command\_Dropped** - A command was dropped due to a full queue.
- **Memory\_Region\_Dropped** - A parameter memory region was dropped due to a full queue.

### 3.9 Data Products

Data products for the Parameter Table Forwarder component.

Table 9: Parameter Table Forwarder Data Products

| Local ID   | Data Product Name | Type                            |
|------------|-------------------|---------------------------------|
| 0x0000 (0) | Table_Status      | Packed_Table_Operation_Status.T |

Data Product Descriptions:

- **Table\_Status** - Version/CRC/timestamp/last-operation status of the active table as known to the forwarder. Emitted on every Set and Validate operation (regardless of outcome). Get\_Copy, Get\_Pointer, and the Dump command do not advance the active-table bookkeeping and therefore do not emit this data product (mirrors the Parameters component).

### 3.10 Data Dependencies

The Parameter Table Forwarder component has no data dependencies.

### 3.11 Packets

Packets for the Parameter Table Forwarder component.

Table 10: Parameter Table Forwarder Packets

| Local ID   | Packet Name       | Type             |
|------------|-------------------|------------------|
| 0x0000 (0) | Active_Parameters | <i>Undefined</i> |

---

Packet Descriptions:

- **Active\_Parameters** - Snapshot of the current parameter table fetched from the downstream component at the time of dump, prefixed with a freshly computed CRC for bit-rot detection.

### 3.12 Faults

The Parameter Table Forwarder component has no faults.

## 4 Unit Tests

The following section describes the unit test suites written to test the component.

### 4.1 *Parameter\_Table\_Forwarder\_Tests* Test Suite

This is a unit test suite for the Parameter Table Forwarder component.

Test Descriptions:

- **Test\_Set\_Success** - A valid Set memory region is forwarded to the downstream which acks Success. Bookkeeping updates, Parameter\_Table\_Updated event fires.
- **Test\_Set\_Length\_Error** - Set with wrong region length is rejected without contacting the downstream.
- **Test\_Set\_Crc\_Error** - Set with mismatched CRC is rejected without contacting the downstream.
- **Test\_Set\_Downstream\_Rejects** - Downstream returns Parameter\_Error on Set. Forwarder propagates and does not update Stored\_Crc.
- **Test\_Validate\_Success** - Validate forwarded; downstream acks Success. Stored\_Crc is NOT updated.
- **Test\_Validate\_Downstream\_Rejects** - Validate forwarded; downstream returns Parameter\_Error.
- **Test\_Validate\_Length\_Crc\_Errors** - Validate rejected for length/CRC mismatches without contacting the downstream.
- **Test\_Get\_Success** - Get forwarded; downstream writes bytes and acks. Forwarder propagates release upstream.
- **Test\_Get\_Length\_Error** - Get region smaller than Table\_Size is rejected without contacting the downstream.
- **Test\_Get\_Downstream\_Rejects** - Downstream returns Parameter\_Error on Get.
- **Test\_Get\_Pointer\_Rejected** - An external Get\_Pointer request is rejected by the forwarder without contacting the downstream; emits Get\_Pointer\_Not\_Supported event and returns Parameter\_Error.
- **Test\_Dump\_Command\_Success** - Dump command issues internal Get; downstream writes bytes; Active\_Parameters packet emitted with CRC prefix.
- **Test\_Dump\_Command\_Failure** - Downstream rejects internal Get during Dump; Dump\_Failed event fires and command response is Failure.
- **Test\_Dump\_Command\_No\_Dump\_Connector** - Dump command with no Memory\_Dump\_Send connector wired; Dump\_Failed event fires with a zero-length sentinel region and the

command response is Failure, without contacting the downstream.

- **Test\_Dump\_On\_Change** - After a successful Set with Dump\_On\_Change=True, an Active\_Parameters packet is auto-emitted.
- **Test\_Dump\_On\_Change\_Failure** - After a successful Set with Dump\_On\_Change=True, the auto-dump's internal Get\_Pointer is rejected; Dump\_Failed event fires and the Set's upstream release reports Parameter\_Error.
- **Test\_Command\_Dropped** - Command queue full triggers Command\_Dropped event.
- **Test\_Memory\_Region\_Dropped** - Memory region queue full triggers Memory\_Region\_Dropped event and a Dropped release upstream.
- **Test\_Initial\_Table\_Status** - Before any operation the Table\_Status DP reports Uninitialized.
- **Test\_Invalid\_Command** - A command with bad length triggers an Invalid\_Command\_Received event.

## 5 Appendix

### 5.1 Preamble

This component contains no preamble code.

### 5.2 Packed Types

The following section outlines any complex data types used in the component in alphabetical order. This includes packed records and packed arrays that might be used as connector types, command arguments, event parameters, etc..

#### Command.T:

Generic command packet for holding arbitrary commands

Table 11: Command Packed Record : 2080 bits (*maximum*)

| Name       | Type                    | Range | Size (Bits) | Start Bit | End Bit | Variable Length          |
|------------|-------------------------|-------|-------------|-----------|---------|--------------------------|
| Header     | Command_Header.T        | -     | 40          | 0         | 39      | -                        |
| Arg_Buffer | Command_Arg_Buffer.Type | -     | 2040        | 40        | 2079    | Header.Arg_Buffer_Length |

Field Descriptions:

- **Header** - The command header
- **Arg\_Buffer** - A buffer that contains the command arguments

#### Command\_Header.T:

Generic command header for holding arbitrary commands

Table 12: Command\_Header Packed Record : 40 bits

| Name              | Type                                                 | Range      | Size (Bits) | Start Bit | End Bit |
|-------------------|------------------------------------------------------|------------|-------------|-----------|---------|
| Source_Id         | Command_Types.<br>Command_Source_Id                  | 0 to 65535 | 16          | 0         | 15      |
| Id                | Command_Types.<br>Command_Id                         | 0 to 65535 | 16          | 16        | 31      |
| Arg_Buffer_Length | Command_Types.<br>Command_Arg_Buffer_<br>Length_Type | 0 to 255   | 8           | 32        | 39      |

Field Descriptions:

- **Source\_Id** - The source ID. An ID assigned to a command sending component.
- **Id** - The command identifier
- **Arg\_Buffer\_Length** - The number of bytes used in the command argument buffer

### Command\_Response.T:

Record for holding command response data.

Table 13: Command\_Response Packed Record : 56 bits

| Name                | Type                                                | Range                                                                                                                                                | Size (Bits) | Start Bit | End Bit |
|---------------------|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|-----------|---------|
| Source_Id           | Command_<br>Types.Command_<br>Source_Id             | 0 to 65535                                                                                                                                           | 16          | 0         | 15      |
| Registration_<br>Id | Command_<br>Types.Command_<br>Registration_<br>Id   | 0 to 65535                                                                                                                                           | 16          | 16        | 31      |
| Command_Id          | Command_Types.<br>Command_Id                        | 0 to 65535                                                                                                                                           | 16          | 32        | 47      |
| Status              | Command_Enums.<br>Command_<br>Response_<br>Status.E | 0 => Success<br>1 => Failure<br>2 => Id_Error<br>3 => Validation_Error<br>4 => Length_Error<br>5 => Dropped<br>6 => Register<br>7 => Register_Source | 8           | 48        | 55      |

Field Descriptions:

- **Source\_Id** - The source ID. An ID assigned to a command sending component.
- **Registration\_Id** - The registration ID. An ID assigned to each registered component at initialization.
- **Command\_Id** - The command ID for the command response.
- **Status** - The command execution status.

### Data\_Product.T:

Generic data product packet for holding arbitrary data types

Table 14: Data\_Product Packed Record : 1096 bits (*maximum*)

| Name   | Type                                        | Range | Size (Bits) | Start Bit | End Bit | Variable Length      |
|--------|---------------------------------------------|-------|-------------|-----------|---------|----------------------|
| Header | Data_Product_Header.T                       | -     | 72          | 0         | 71      | -                    |
| Buffer | Data_Product_Types.Data_Product_Buffer_Type | -     | 1024        | 72        | 1095    | Header.Buffer_Length |

Field Descriptions:

- **Header** - The data product header
- **Buffer** - A buffer that contains the data product type

### Data\_Product\_Header.T:

Generic data\_product packet for holding arbitrary data\_product types

Table 15: Data\_Product\_Header Packed Record : 72 bits

| Name          | Type                                               | Range      | Size (Bits) | Start Bit | End Bit |
|---------------|----------------------------------------------------|------------|-------------|-----------|---------|
| Time          | Sys_Time.T                                         | -          | 48          | 0         | 47      |
| Id            | Data_Product_Types.Data_Product_Id                 | 0 to 65535 | 16          | 48        | 63      |
| Buffer_Length | Data_Product_Types.Data_Product_Buffer_Length_Type | 0 to 128   | 8           | 64        | 71      |

Field Descriptions:

- **Time** - The timestamp for the data product item.
- **Id** - The data product identifier
- **Buffer\_Length** - The number of bytes used in the data product buffer

### Event.T:

Generic event packet for holding arbitrary events

Table 16: Event Packed Record : 328 bits (*maximum*)

| Name         | Type                              | Range | Size (Bits) | Start Bit | End Bit | Variable Length            |
|--------------|-----------------------------------|-------|-------------|-----------|---------|----------------------------|
| Header       | Event_Header.T                    | -     | 72          | 0         | 71      | -                          |
| Param_Buffer | Event_Types.Parameter_Buffer_Type | -     | 256         | 72        | 327     | Header.Param_Buffer_Length |

Field Descriptions:

- **Header** - The event header
- **Param\_Buffer** - A buffer that contains the event parameters

### Event\_Header.T:

Generic event packet for holding arbitrary events

Table 17: Event\_Header Packed Record : 72 bits

| Name                | Type                                     | Range      | Size (Bits) | Start Bit | End Bit |
|---------------------|------------------------------------------|------------|-------------|-----------|---------|
| Time                | Sys_Time.T                               | -          | 48          | 0         | 47      |
| Id                  | Event_Types.Event_Id                     | 0 to 65535 | 16          | 48        | 63      |
| Param_Buffer_Length | Event_Types.Parameter_Buffer_Length_Type | 0 to 32    | 8           | 64        | 71      |

Field Descriptions:

- **Time** - The timestamp for the event.
- **Id** - The event identifier
- **Param\_Buffer\_Length** - The number of bytes used in the param buffer

### Invalid\_Command\_Info.T:

Record for holding information about an invalid command

Table 18: Invalid\_Command\_Info Packed Record : 112 bits

| Name                | Type                     | Range           | Size (Bits) | Start Bit | End Bit |
|---------------------|--------------------------|-----------------|-------------|-----------|---------|
| Id                  | Command_Types.Command_Id | 0 to 65535      | 16          | 0         | 15      |
| Errant_Field_Number | Interfaces.Unsigned_32   | 0 to 4294967295 | 32          | 16        | 47      |
| Errant_Field        | Basic_Types.Poly_Type    | -               | 64          | 48        | 111     |

Field Descriptions:

- **Id** - The command Id received.
- **Errant\_Field\_Number** - The field that was invalid. 1 is the first field, 0 means unknown field, 2\*\*32 means that the length field of the command was invalid.
- **Errant\_Field** - A polymorphic type containing the bad field data, or length when Errant\_Field\_Number is 2\*\*32.

### Invalid\_Parameters\_Memory\_Region\_Crc.T:

A packed record which holds data related to an invalid parameter memory region CRC.

Table 19: Invalid\_Parameters\_Memory\_Region\_Crc Packed Record : 168 bits

| Name              | Type                       | Range | Size (Bits) | Start Bit | End Bit |
|-------------------|----------------------------|-------|-------------|-----------|---------|
| Parameters_Region | Parameters_Memory_Region.T | -     | 104         | 0         | 103     |
| Header            | Parameter_Table_Header.T   | -     | 48          | 104       | 151     |
| Computed_Crc      | Crc_16.Crc_16_Type         | -     | 16          | 152       | 167     |

Field Descriptions:

- **Parameters\_Region** - The memory region and operation.
- **Header** - The parameter table header stored in the memory region.
- **Computed\_Crc** - The FSW computed CRC of the parameter table stored in the memory region.

### Invalid\_Parameters\_Memory\_Region\_Length.T:

A packed record which holds data related to an invalid parameter memory region length.

Table 20: Invalid\_Parameters\_Memory\_Region\_Length Packed Record : 136 bits

| Name              | Type                       | Range           | Size (Bits) | Start Bit | End Bit |
|-------------------|----------------------------|-----------------|-------------|-----------|---------|
| Parameters_Region | Parameters_Memory_Region.T | -               | 104         | 0         | 103     |
| Expected_Length   | Natural                    | 0 to 2147483647 | 32          | 104       | 135     |

Field Descriptions:

- **Parameters\_Region** - The memory region and operation.
- **Expected\_Length** - The length bound that the memory region failed to meet.

### Memory\_Region.T:

A memory region described by a system address and length (in bytes).

Table 21: Memory\_Region Packed Record : 96 bits

| Name    | Type           | Range           | Size (Bits) | Start Bit | End Bit |
|---------|----------------|-----------------|-------------|-----------|---------|
| Address | System.Address | -               | 64          | 0         | 63      |
| Length  | Natural        | 0 to 2147483647 | 32          | 64        | 95      |

Field Descriptions:

- **Address** - The starting address of the memory region.
- **Length** - The number of bytes at the given address to associate with this memory region.

### Packed\_Table\_Operation\_Status.T:

A packed record containing the parameter table operation status information.

Table 22: Packed\_Table\_Operation\_Status Packed Record : 88 bits

| Name                        | Type                    | Range                       | Size (Bits) | Start Bit | End Bit |
|-----------------------------|-------------------------|-----------------------------|-------------|-----------|---------|
| Active_Table_Version_Number | Short_Float             | -3.40282e+38 to 3.40282e+38 | 32          | 0         | 31      |
| Active_Table_Update_Time    | Interfaces0 Unsigned_32 | 0 to 4294967295             | 32          | 32        | 63      |

|                                        |                                                                       |                                                                                                                                                         |    |    |    |
|----------------------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|----|----|----|
| Active_<br>Table_<br>Crc               | Crc_16.<br>Crc_16_<br>Type                                            | -                                                                                                                                                       | 16 | 64 | 79 |
| Last_<br>Table_<br>Operation<br>Status | Parameter<br>Enums.<br>Parameter<br>Table_<br>Update_<br>Status.<br>E | 0 => Uninitialized<br>1 => Success<br>2 => Length_Error<br>3 => Crc_Error<br>4 => Parameter_Error<br>5 => Dropped<br>6 => Individual_Parameter_Modified | 8  | 80 | 87 |

Field Descriptions:

- **Active\_Table\_Version\_Number** - The version number of the active parameter table.
- **Active\_Table\_Update\_Time** - The timestamp (in seconds) of the last parameter table operation.
- **Active\_Table\_Crc** - The CRC of the active parameter table.
- **Last\_Table\_Operation\_Status** - The status of the last parameter table operation.

### Parameter\_Table\_Header.T:

A packed record which holds parameter table header data. This data will be prepended to the table data upon upload. *Preamble (inline Ada definitions):*

```

1  -- Declare the start index at which to begin calculating the CRC. The
2  -- start index is dependent on this type, and so is declared here so that
3  -- it is easier to keep in sync.
4  Crc_Section_Length : constant Natural := Crc_16.Crc_16_Type'Length;
5  Version_Length : constant Natural := 4;

```

Table 23: Parameter\_Table\_Header Packed Record : 48 bits

| Name      | Type               | Range                       | Size<br>(Bits) | Start<br>Bit | End<br>Bit |
|-----------|--------------------|-----------------------------|----------------|--------------|------------|
| Crc_Table | Crc_16.Crc_16_Type | -                           | 16             | 0            | 15         |
| Version   | Short_Float        | -3.40282e+38 to 3.40282e+38 | 32             | 16           | 47         |

Field Descriptions:

- **Crc\_Table** - The CRC of the parameter table, as computed by a ground system, and uplinked with the table.
- **Version** - The current version of the parameter table.

### Parameters\_Memory\_Region.T:

A packed record which holds the parameter memory region to operate on as well as an enumeration specifying the operation to perform.

Table 24: Parameters\_Memory\_Region Packed Record : 104 bits

| Name      | Type                                                     | Range                                                          | Size (Bits) | Start Bit | End Bit |
|-----------|----------------------------------------------------------|----------------------------------------------------------------|-------------|-----------|---------|
| Region    | Memory_Region.T                                          | -                                                              | 96          | 0         | 95      |
| Operation | Parameter_Enums.<br>Parameter_Table_<br>Operation_Type.E | 0 => Get_Copy<br>1 => Get_Pointer<br>2 => Set<br>3 => Validate | 8           | 96        | 103     |

Field Descriptions:

- **Region** - The memory region.
- **Operation** - The parameter table operation to perform.

### Parameters\_Memory\_Region\_Release.T:

A packed record which holds the parameter memory region to release as well as the status returned from the parameter update operation.

Table 25: Parameters\_Memory\_Region\_Release Packed Record : 104 bits

| Name   | Type                                                | Range                                                                                                                                                   | Size (Bits) | Start Bit | End Bit |
|--------|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|-----------|---------|
| Region | Memory_Region.T                                     | -                                                                                                                                                       | 96          | 0         | 95      |
| Status | Parameter_Enums.<br>Parameter_Table_Update_Status.E | 0 => Uninitialized<br>1 => Success<br>2 => Length_Error<br>3 => Crc_Error<br>4 => Parameter_Error<br>5 => Dropped<br>6 => Individual_Parameter_Modified | 8           | 96        | 103     |

Field Descriptions:

- **Region** - The memory region.
- **Status** - The return status from the parameter update operation.

### Sys\_Time.T:

A record which holds a time stamp using EMA VTC format including seconds and subseconds since epoch (1-5-1980 to 1-6-1980 midnight).

Table 26: Sys\_Time Packed Record : 48 bits

| Name       | Type                   | Range           | Size (Bits) | Start Bit | End Bit |
|------------|------------------------|-----------------|-------------|-----------|---------|
| Seconds    | Interfaces.Unsigned_32 | 0 to 4294967295 | 32          | 0         | 31      |
| Subseconds | Interfaces.Unsigned_16 | 0 to 65535      | 16          | 32        | 47      |

---

Field Descriptions:

- **Seconds** - The number of seconds elapsed since epoch.
- **Subseconds** - The number of  $1/(2^{16})$  sub-seconds.

### 5.3 Enumerations

The following section outlines any enumerations used in the component.

#### **Command\_Enums.Command\_Response\_Status.E:**

This status enumeration provides information on the success/failure of a command through the command response connector.

Table 27: Command\_Response\_Status Literals:

| Name             | Value | Description                                                                                                         |
|------------------|-------|---------------------------------------------------------------------------------------------------------------------|
| Success          | 0     | Command was passed to the handler and successfully executed.                                                        |
| Failure          | 1     | Command was passed to the handler not successfully executed.                                                        |
| Id_Error         | 2     | Command id was not valid.                                                                                           |
| Validation_Error | 3     | Command parameters were not successfully validated.                                                                 |
| Length_Error     | 4     | Command length was not correct.                                                                                     |
| Dropped          | 5     | Command overflowed a component queue and was dropped.                                                               |
| Register         | 6     | This status is used to register a command with the command routing system.                                          |
| Register_Source  | 7     | This status is used to register command sender's source id with the command router for command response forwarding. |

#### **Parameter\_Enums.Parameter\_Table\_Update\_Status.E:**

This status enumeration provides information on the success/failure of a parameter table update.

Table 28: Parameter\_Table\_Update\_Status Literals:

| Name            | Value | Description                                                                                                                       |
|-----------------|-------|-----------------------------------------------------------------------------------------------------------------------------------|
| Uninitialized   | 0     | No table operation has been performed yet.                                                                                        |
| Success         | 1     | Parameter table update operation was successful.                                                                                  |
| Length_Error    | 2     | Parameter table length was not correct.                                                                                           |
| Crc_Error       | 3     | The computed CRC of the table does not match the stored CRC.                                                                      |
| Parameter_Error | 4     | An individual parameter was found invalid due to a constraint error within a component, or failing component-specific validation. |

|                               |   |                                                                                                                                                                                                                                                                                                                                                                            |
|-------------------------------|---|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dropped                       | 5 | The operation could not be performed because it was dropped from a full queue.                                                                                                                                                                                                                                                                                             |
| Individual_Parameter_Modified | 6 | An individual parameter was updated in the parameter table without updating the whole table. In this case, the reported CRC, version, and update time of the table can no longer be trusted to be accurate. This status will be reported when using the Update_Parameter command in the Parameters component, which is intended to be a backdoor, ground-use only command. |

### Parameter\_Enums.Parameter\_Table\_Operation\_Type.E:

This enumeration lists the different parameter table operations that can be performed.

Table 29: Parameter\_Table\_Operation\_Type Literals:

| Name        | Value | Description                                                                                                                                                                                                                                                    |
|-------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Get_Copy    | 0     | Retrieve the current values of the parameters by copying them into a caller-provided memory region.                                                                                                                                                            |
| Get_Pointer | 1     | Retrieve the current values of the parameters by returning a memory region pointing at the owner's buffer (zero-copy). The caller-provided region in the request is ignored. The responder fills the returned release with a region pointing at its own bytes. |
| Set         | 2     | Set the current values of the parameters.                                                                                                                                                                                                                      |
| Validate    | 3     | Validate the current values of the parameters.                                                                                                                                                                                                                 |